Fractal Continual Learning: Recursive Structural Growth, Long-Horizon Adaptation, and the Boundary of Structural Advantage

HUSHFUSION Forge Research

October 2026

Abstract

Continual learning is usually formulated as parameter adaptation under a fixed computational graph. We study a stronger question: can the learning graph itself grow recursively while continuing to adapt online? We introduce a minimal fractal-loop architecture in which a dense root loop and sparse descendant loops communicate through normalized convex gates. A relative-decay detector grows descendants when the current improvement rate falls below a fraction of the structure's early improvement rate; descendants update at a frequency reduced by the fractal depth. We evaluate the method in a non-stationary control environment derived from a two-flow-ring μ-dynamics model and compare it with a fixed single-node learner. A 72,335-round run shows that both systems learn continuously under drifting initial conditions, reducing warm-control error from approximately 130–176 steps to approximately 9 steps while maintaining a cold random baseline near 32 steps. The fractal tree grows to seven nodes and then stabilizes. However, the fractal tree does not outperform the fixed learner in the small-model regime: after convergence, paired warm performance differs by less than 0.3 steps and the exceedance rates are statistically indistinguishable at the resolution of this experiment. Cross-validation on 701 real Forge configuration–score records likewise finds no fractal advantage. The result is therefore not a claim that fractal structure universally improves continual learning. It is a boundary result: recursive structural growth is implementable and stable over long horizons, but its performance advantage requires a regime with sufficient scale and diminishing marginal returns. We release the protocol, negative results, and failure analyses to make this boundary reproducible.

1 Introduction

Continual learning systems typically update parameters while holding depth, connectivity, and the division between fast adaptation and slow consolidation fixed. This separation is useful, but it leaves a structural question unanswered: when the task changes, should the learner only change its weights, or should it also change the organization of the loops that update those weights? The question matters in non-stationary control, where a learner must track drifting conditions while preserving a stable response mechanism.

We make three claims. First, a recursive structural learner can grow additional loop nodes from a single root using an online improvement signal, communicate across levels through a normalized convex gate, and stabilize rather than grow without bound. Second, this mechanism can continue learning for tens of thousands of rounds under drifting control conditions. Third, in the small-data, small-model regime studied here, structural growth does not produce a measurable advantage over

a fixed learner. The third result is deliberate: a negative structural comparison is more useful than a universal claim unsupported by the data.

This paper is about the engineering and empirical boundary of recursive continual learning, not a new physical derivation of antigravity. The control environment is assembled from an existing μ-dynamics model; the learner optimizes control choices inside that model. We use the environment because it provides a deterministic, inspectable, non-stationary benchmark with a safety ceiling, not because the experiment establishes that the underlying physical model is experimentally confirmed.

The rest of the paper first defines the recursive loop architecture and its growth rule, then describes the control environment and the paired long-horizon protocol. We report the primary run, multi-seed and clean-evaluation checks, compare the result with the structural predictions motivating fractal allocation [5], and finish with limitations and the conditions under which a structural advantage should be expected.

The loop design draws on established meta-learning (fast task adaptation separated from slower meta-updates [1, 3]), continual-learning consolidation motives [2], and fractal allocation ideas formalized in our companion Lean development [5]. Our contribution is not a new optimizer; it is a concrete, reproducible protocol for asking whether structural growth itself—rather than weight growth alone—improves long-horizon adaptation, together with an honest boundary result.

2 Related Structural Ideas

The design is motivated by three families of ideas. Optimization-based meta-learning separates fast task adaptation from slower meta-parameter updates [1, 3]; our root and descendant loops preserve this distinction while allowing the graph of update mechanisms to change. Residual and contraction arguments motivate stable information transfer between levels. Finally, fractal allocation arguments, which echo Murray's minimum-work principle for branching networks [6], suggest that when marginal returns decline with depth, computation need not be distributed uniformly across all levels [5]. These ideas motivate our architecture, but none of them by itself predicts an empirical win in the present small regime.

A useful distinction is between a structural mechanism and a structural advantage. The former asks whether a graph can grow, communicate, and remain trainable. The latter asks whether that graph improves a specified metric under a matched compute and data budget. This paper treats the two questions separately.

3 Recursive Fractal Loop Architecture

3.1 Loop tree

Let each loop node v contain a predictor $f_v(x;\theta_v)$. A node may have descendants $\mathcal{C}(v)$. The node output is

$$y_v(x) = g_{v,0} f_v(x;\theta_v) + \sum_{u\in\mathcal{C}(v)} g_{v,u} y_u(x), \qquad g_{v,j} \geq 0, \quad \sum_j g_{v,j} = 1, \tag{1}$$

where the gate vector is produced by a softmax. The convex combination gives a bounded communication interface between levels. A descendant is not automatically trusted: it is first warmed independently, after which its output can enter the parent's mixture.

3.2 Fast and slow updates

The root is updated every round. A node at depth d is updated at frequency proportional to D^{-d}, where D is the sparse scaling factor. Thus shallow loops receive dense updates and deeper loops receive sparse updates. This is the operational analogue of a fractal allocation rule: computation is concentrated near the prefix where marginal learning value is highest, while deeper structure is available for specialization.

3.3 Growth rule

The growth signal is the relative decay of validation loss. If the exponentially weighted improvement rate in the current window falls below a fraction of the early improvement rate for several consecutive observations, the node spawns a descendant. The detector is reset after growth. A clean-loss filter excludes safety-violation penalty values from the growth signal. This distinction is important: a penalty spike is not evidence that the representation has reached a structural bottleneck.

Definition 1 (Structural growth)**.** *A growth event is a graph mutation that adds a descendant node, initializes its parameters, warms it independently, and then exposes it to the parent through a normalized gate.*

The mechanism is intentionally minimal. It does not claim that the chosen detector is optimal. In particular, a first attempt based on an absolute "loss no longer decreases" rule failed because shallow networks can improve forever at a very slow rate. Relative decay was introduced to detect loss of improvement relative to the structure's own early behavior.

4 Control Environment

We use a deterministic μ-dynamics environment assembled from the two-flow-ring control model. The state update is

$$\mu_{t+1} = \mu_t + \eta_t(1 - \mu_t), \qquad \eta_t = \eta_{\text{ext}} + (2\lambda - \lambda^2)(1 - \mu_t), \tag{2}$$

where η_{ext} is an external drive and λ is a convergence parameter controlling the candidate self-flattening term. The task is to reach $\mu \geq 0.999$ without crossing the ceiling $\mu_{\text{ceil}} = 1 - m_e/m_i \approx 0.999780568$. A trial is scored by its number of steps to the work window; ceiling crossing is a safety failure.

This environment is suitable for method study because it is deterministic, has a clear safety boundary, and can be made non-stationary by sampling a different initial state μ_0 each round. It is not evidence that an experimental antigravity device exists or that λ is physically available. In the upstream model, λ remains an input gap.

5 Experimental Protocol

5.1 Paired long-horizon evaluation

At every round we present the same μ_0 to a fixed single-node learner and to the fractal learner. Each learner selects a control point using its own accumulated experience, executes the environment, and updates its own parameters. Odd rounds are cold random controls and even rounds are warm recommendations. The paired design removes the largest source of confounding: the two structures see the same drifting condition.

Table 1: Long-horizon paired result.

Metric	Fixed	Fractal
Warm mean steps	11.1	11.5
Cold mean steps	31.6	31.9
Exceedance rate	5.9%	5.8%
Final nodes	1	7

The deployed run used 72,335 rounds over approximately 24 hours, with independent checkpoints, a sliding experience window, and a supervisor that restarted the daemon after failure. The fractal tree was allowed to grow without a preselected target depth, subject to a safety cap. The final tree contained seven nodes.

5.2 Secondary checks

We also performed three secondary checks. First, three-seed tests examined whether a short-run fractal advantage was robust. Second, clean cross-validation compared fixed and fractal predictors on 701 real Forge configuration–score records. Third, earlier controlled spikes tested the growth detector, the gate initialization, and the effect of separating safety classification from score regression.

6 Results

6.1 Long-horizon run

Both systems learned substantially. Warm performance fell from roughly 130–176 steps during the initial phase to roughly 9 steps after convergence, while the cold random baseline remained near 32 steps. Thus continual online adaptation was effective and remained stable over tens of thousands of rounds.

After convergence, the fixed learner achieved 11.1 warm steps averaged over the full run and the fractal learner achieved 11.5. Restricting attention to the converged region, both remained near 9 steps. Paired warm outcomes were fixed-winner 16.1%, fractal-winner 20.9%, and ties approximately 63%. This apparently favorable fractal win rate did not translate into a lower mean; the difference is within the variability of paired outcomes. Safety exceedance was 5.9% for fixed and 5.8% for fractal.

6.2 Structural advantage and its failure boundary

The structural mechanism worked: the tree grew automatically and stabilized. The performance claim did not: seven nodes did not improve the final metric. This is a useful separation. Growth is not evidence of advantage; it is evidence that the graph mutation and communication mechanisms are operational.

The multi-seed short-run experiment did not support a reliable advantage. One seed favored fractal growth, one was approximately tied, and one favored the fixed learner. The clean real-data experiment was consistent with the long run: a fixed node achieved five-fold MSE 0.0719 ± 0.0468 after longer training, while a fractal tree with three descendants achieved 0.0861 ± 0.0536. The difference is not a proof that fractal organization is harmful in general; it shows that this regime does not provide the conditions under which the structural hypothesis should win.

7 Failure Analysis

Several negative results were important for the final design. An absolute non-decreasing-loss detector did not grow because a shallow model continued making small improvements indefinitely. A relative-decay detector grew reliably. A first fractal implementation initialized child gates at zero; child outputs then had zero contribution and effectively zero gate gradient. Uniform gate initialization plus a warm-up phase repaired the dead-child mechanism. A separate failure occurred at the OML interface: adding a layer without expanding and reinitializing its per-parameter step-size meta-parameters damaged the fast adaptation loop. Structural growth and meta-parameter growth therefore cannot be treated as independent operations.

Finally, an RSI-style technical indicator was tested as a growth signal. Standard RSI is inappropriate for a monotonically improving loss trajectory: when the improvement deltas are one-sided, losses are zero and RSI saturates at 100. We therefore interpret "RSI" here as recursive self-improvement rather than as a financial indicator. The adaptive threshold experiment showed that growth thresholds can themselves be updated from the measured value of a structural mutation, but this meta-controller did not outperform a hand-tuned threshold in the small toy regime.

8 Discussion

The experiment supports a modest but concrete claim. A continual learner can maintain a fixed root loop, grow recursively generated descendants, communicate through convex gates, and continue learning under a drifting environment for tens of thousands of rounds. This is a model-level form of plasticity, distinct from merely updating weights.

The experiment does not support the stronger claim that recursive fractal structure is generally more accurate or efficient. In the present regime, the additional nodes were a complexity tax. The most plausible explanation is scale: with small models and limited data, marginal returns do not decline in the way required by prefix-concentration arguments, and splitting the learning budget across specialists only dilutes evidence. The hypothesis should be revisited with larger networks, more domains, and tasks whose representation genuinely has multi-scale structure.

The practical lesson is methodological. Structural growth should be evaluated in three separate stages: (i) does the graph mutate safely, (ii) does the learner continue adapting after mutation, and (iii) does the mutation improve a pre-registered metric under matched compute and data? Conflating these stages turns a working plasticity mechanism into an unsupported performance claim.

9 Conclusion

We introduced and tested a minimal fractal continual-learning architecture with recursively growing loops, sparse deep updates, and convex inter-level communication. The architecture grew to seven nodes and remained stable for 72,335 rounds in a drifting control environment. Both fixed and fractal learners achieved sustained continual improvement, but the fractal learner did not outperform the fixed learner in the small-model regime, in multi-seed checks, or on 701 real configuration–score records. The result establishes a reproducible boundary: recursive structural plasticity is feasible and useful as a model of long-horizon adaptation, but a performance advantage requires a larger and more structurally demanding regime.

Data and code availability

The implementation, supervisors, long-horizon JSONL traces, checkpoints, and spike scripts are maintained in the HUSHFUSION Forge and headless research repositories [4]. The reported long-horizon trace contains 72,335 paired rounds. All claims in this paper are restricted to the model and protocol described here.

References

[1] Chelsea Finn, Pieter Abbeel, and Sergey Levine. Model-agnostic meta-learning for fast adaptation of deep networks. In *Proceedings of the 34th International Conference on Machine Learning (ICML)*, pages 1126–1135, 2017.

[2] James Kirkpatrick, Razvan Pascanu, Neil Rabinowitz, Joel Veness, Guillaume Desjardins, Andrei A. Rusu, Kieran Milan, John Quan, Tiago Ramalho, Agnieszka Grabska-Barwinska, et al. Overcoming catastrophic forgetting in neural networks. *Proceedings of the National Academy of Sciences*, 114(13):3521–3526, 2017.

[3] Zhenguo Li, Fengwei Zhou, Fei Chen, and Hang Li. Meta-SGD: Learning to learn quickly for few-shot learning. *arXiv preprint arXiv:1707.09835*, 2017.

[4] Yuanjie Liu. HUSHFUSION Forge: design search, knowledge reuse, and continual-learning daemons. `https://github.com/logos-42/hushfusion-forge`, 2026. Long-horizon fracta-loop trace and reproduce scripts.

[5] Yuanjie Liu. LmPrinciple: Fractal allocation theorems for diminishing-return structures. `https://github.com/logos-42/lm-principle`, 2026. Lean formalization: Fractal.lean, prefix_allocation_optimal, fractal_beats_uniform.

[6] Cecil D. Murray. The physiological principle of minimum work: I. The vascular system and the cost of blood volume. *Proceedings of the National Academy of Sciences*, 12(3):207–214, 1926.